

Day 59 -- Line-by-Line Syntax Explanation

day59_vanishing_exploding_gradients.py, segment by segment | MD Mutasim Billah

1. The Task and Spectral Radius

make_sign_task: a label that depends on one ordinary-looking input

IMPLEMENTATION

```
X = rng.randn(n, T)          # (n, T) matrix of N(0, 1) draws
y = (X[:, 0] > 0).astype(int) # X[:, 0] is column 0: every sequence's first input
```

`X[:, 0] > 0` produces a boolean array; `.astype(int)` turns True/False into 1/0 labels. Because `x_0` is drawn from the same distribution as every other input, no magnitude cue marks it as the important one.

spectral radius and rescaling to a target

IMPLEMENTATION

```
np.max(np.abs(np.linalg.eigvals(W))) # eigenvalues can be complex -> np.abs gives modulus
net.Whh *= rho / spectral_radius(net.Whh) # in-place scale: new radius = rho exactly
```

`np.linalg.eigvals` returns possibly-complex eigenvalues of a square matrix; `np.abs` takes each one's magnitude. Multiplying a matrix by `c` multiplies every eigenvalue by `c`, so scaling by `rho / (current radius)` lands exactly on `rho`.

2. The Probe

the true gradient of the MEAN loss

IMPLEMENTATION

```
dlogits = (net.probs - one_hot(y, 2)) / m
```

Day 58's `backward()` divides by `m` later, inside each weight gradient. The probe divides immediately, so every `dL/dh_t` it records is the real derivative of the averaged loss -- which is what the numeric check measures.

recording while walking back

IMPLEMENTATION

```
for t in reversed(range(T)):
    dH[:, t, :] = dh          # store BEFORE transforming
    da = dh * (1.0 - net.H[:, t, :] ** 2)
    dh = da @ net.Whh.T
```

The store must happen before `dh` is transformed: at loop entry, `dh` is exactly `dL/dh_t`. After the two lines below it, `dh` has become `dL/dh_{t-1}`, ready for the next iteration.

the Frobenius norm per timestep

IMPLEMENTATION

```
np.array([np.linalg.norm(dH[:, t, :]) for t in range(dH.shape[1])])
```

`np.linalg.norm` on a 2-D array defaults to the Frobenius norm -- the square root of the sum of all squared entries -- giving one number per timestep that summarizes the whole batch.

3. Tracking a Training Run

the global gradient norm

IMPLEMENTATION

```
np.sqrt(sum(np.sum(g ** 2) for g in net.grads.values()))
```

`net.grads` is a dict of five arrays of different shapes. The generator expression squares and sums each one, `sum()` adds the five totals, and the square root gives the length of the gradient as if all parameters were one long vector -- the quantity gradient clipping will cap on Day 60.

finding spikes and what they caused

IMPLEMENTATION

```
spikes = np.where(g > 10 * med)[0]  
dL = np.abs(np.diff(L))  
after = dL[spikes[spikes < len(dL)]]
```

`np.where` on a boolean array returns a tuple of index arrays; `[0]` takes the one for this 1-D case. `np.diff` gives `L[i+1] - L[i]`, so `dL[s]` is the loss change caused by step `s`'s update. The guard `spikes < len(dL)` drops a spike on the very last step, which has no 'next' loss.

the influence check

IMPLEMENTATION

```
X_flip = X_te.copy(); X_flip[:, 0] *= -1  
X_mid = X_te.copy(); X_mid[:, T // 2] = np.random.RandomState(5).randn(len(y_te))
```

`.copy()` matters: without it, `X_flip` would be a second name for `X_te` and the flip would corrupt the baseline. `T // 2` is integer division -- the middle timestep. `*= -1` negates column 0 in place, flipping every sequence's label.